

机器与连接（草案） + 诊断附录

机器与连接

这一页讲清楚话题、席位、会话、工作机分别是什么，模型调用和文件命令各在哪台机器上跑，以及连接出问题怎么排查。

四个概念

这四个词在其他页面经常一起出现，先各给一句定义。

话题

话题是一个协作房间：人和 AI 队友在里面发消息、派任务、看进度。每个话题有自己的文件和成果。

席位

席位是房间里的一个位置，坐着一个人或一个 AI 队友。房间顶部的成员头像就是名册上的席位。

会话

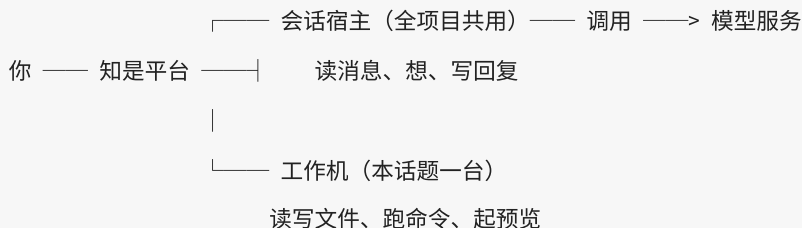
会话是 AI 队友席位上正在跑的那段工作：它记得这个队友读到了哪、正在做什么。会话跑在平台共用的会话宿主上，不在你的工作机上。

工作机

工作机是读写文件、跑命令、打开预览的那台机器。一个话题一台，房间里所有 AI 队友共用；可以是云端机器，也可以是接入的自有设备。

模型和文件各跑在哪

AI 队友的「想」和「动手」在两个地方，用一张图分清：



发消息后的第一条回复来自会话宿主；工具结果（改了什么文件、命令输出）来自工作机。

三种「卡」对应三个地方

「卡」先分是哪一种，再决定找谁。

- 发消息后第一条回复慢：可能在排队（项目并发上限）、云端工作机准备、会话宿主或模型服务。
- 回复正常、跑命令和读写文件慢：工作机。
- 预览打不开：分「应用没在跑」和「隧道断开」两层，对照下面「预览打不开」处理。

连接出问题时

按现象对照处理：先照抄提示原文，再看意思和处理。

「云端工作电脑正在准备并接入」

意思：云端工作机在开机接入，对话和平台工具仍可用。冷启动可能要两三分钟。处理：先等。超过约 15 分钟（经验值，不是平台承诺）还没好：名册里有另一台不同的可用工作电脑或配置，才换（步骤见[设备与工作电脑](#)）；重选同一个云端配置不会重建卡住的机器。没有别的可选，就截图话题状态和准备记录反馈，不要反复强换。

设备显示「离线」

意思：自有设备与平台断开。处理：按[常见问题与排障](#)里的设备离线步骤处理。云端工作机状态异常就在名册里换机。

「对话太长，正在整理上下文；整理完会接着处理，期间不会回复」

意思：AI 队友的会话在整理（压缩）长对话，发生在会话宿主，和工作机无关；整理期间不回复是设计如此。对话很长时可能反复出现。处理：看这条消息本身：整理中它会被原位更新，更新时刻在往前走说明仍有活动。变成「上下文已整理，接着处理」就继续；变成「上下文整理没有完成」就截图反馈。长时间没有新更新，截图反馈，带上话题、提示的更新时刻、最后一条实际命令。状态接口读到空值不证明

会话结束——平台对会话内部状态的观测目前有缺口，不要因此换机或重启；整理发生在会话宿主，换机也帮不上。

预览打不开

意思：预览分两层：应用（AI 队友跑的 dev server）和隧道（它与平台的连接）。话题面板会分开报，「通道在，应用没在跑」就是隧道正常、应用没起。预览地址只由话题和平台域名决定，换机、重开面板都不会换地址。处理：先看面板报的是哪一层。应用没跑：让声明预览的那个 AI 队友把预览服务重新跑起来。刚换过机：预览服务不会自己跟到新机器，让 AI 在新机上重启原来的预览服务，再打开话题预览。重开面板只是查询，不会替你启动服务。

只有某条命令慢

意思：通常是这条命令本身吃资源。装依赖、跑全量测试会把内存盘占满，可能把机器打到离线。处理：先看任务在跑什么；大命令跑完前别叠加新的大命令。

默认用哪台

没选过工作机的话题按顺序继承：项目默认（「项目设置 → 工作电脑」）→ 团队默认 → 平台默认。改默认只影响还没开工的 AI 队友，正在跑的不受影响。每个 AI 队友现在用哪台，写在房间名册里。选择和更换的操作步骤见[设备与工作电脑](#)。

换机影响整个话题

换机对整个话题生效，不是只换某一个席位。分两种操作。

话题内换机（名册或话题设置）：整个房间一起搬。

1. 每条会话先把自己没推送的改动推上各自的分支。
2. 全部搬成才改房间的选择记录；一条推不上去，整个房间不换，已经搬动的会自己走回来。
3. 原来那台够不着时，平台会问你要不要「不推送，直接更换」：没推上去的改动不会跟过去。

项目设备页的批量切换：正在跑任务的房间整个跳过，不会把它手上的机器抽走。

注意两点。重选同一个配置不算换机，平台当成没变；要换就得选不同的机器或配置。刚换完的一小段时间里，名册可能暂时显示不同的机器，那是搬迁的中间态，稍等刷新。话题归档后，平台按清理条件异步释放云端工作机（释放前先检查发布、活动轮次和目录）。

资源边界

云端工作机是共享宿主机上的容器，核数和内存由建话题时选的机型决定（档位表待补，见文末「还没核实的」）。宿主机上的其他负载可能间接影响你的速度。容器里看到的部分系统读数（负载、磁盘）反映的可能是整台宿主机，不只是这个话题。自有设备的资源边界就是你那台机器本身。

这一版还没核实的

以下几条在这版草案里还没有权威来源，先标出来。

- 各机型的核数、内存档位表：待产品确认后补。
- 「超过约 15 分钟换机」的 15 分钟是经验值，不是平台承诺。
- 会话宿主和模型服务的速度、排队指标目前没有用户可读的看板；觉得整个平台都慢时，截图话题状态找管理员。
- 本文界面字样核对于 2026-09-30 的 dev 环境，以当期产品为准。

诊断附录：原始读数与未决项

本附录附在《机器与连接》草案之后，给核对数据用。所有读数注明窗口、来源、单位、范围；测不到的标 unknown，不补猜。

代码基准

- 采样时部署 SHA：9d035d8353b36d3ccd09e202cdd7d4a4bd523af7（2026-09-30 09:08Z 由 /api/health 的 data.version 读得，当时与 main 一致，对应 PR #2216，前端改动）。之后是否再部署未跟踪。
- 本地比对：/tmp/cheese-src 浅克隆 b0aa4cd..origin/main 对本文引用的 14 个文件 `git diff` 为零（比对时刻 2026-09-30 上午），代码结论在上述 SHA 上成立。
- 2026-09-30 上午三次部署：进程起于 08:03:37Z、约 08:25–08:50、08:58:24Z（uptime 反推 + version 变化确认）。进程内指标随部署清零。

采样机与窗口

- 机器：本话题云端工作机（LXC 容器，pve119 宿主），8 核 cpuset（25,31,34,49,70,77,103,120），窗口约 2026-09-30 07:30–09:30Z 内的多次短采样。
- 采样方法：cgroup v2 命名空间根即容器根；cpu.stat 差分得单核占比；memory 读 anon 不读 current（file page cache 可回收）。

四层读数（示例，不代表平台）

层	读数	窗口	单位
cheese.service 子树 CPU	约 2.3% 单核	5 分钟差分	单核占比
容器可见总量 CPU	轻载	同上	单核占比
宿主 /proc/loadavg	≈35	瞬读	宿主整机背景，与本容器无关
宿主侧 LXC 限额	unknown	—	容器内不可见

容器可见层级 cpu.max/memory.max/pids.max 全为 max，不证明宿主没限；io 控制器未委派，容器级磁盘 MB/s 测不到（unknown）。

admin 读数（wangchangxin 以管理员身份取）

指标	读数	时刻	语义边界
performance uptime	149.47s / 576.075s	约 08:06Z / 约 09:06Z	单进程，随部署清零
有样本路由	31 / 48（注册 685）	同上	中间件计数
active_requests	7 / 5	同上	call_next 返回后减，流式不在账
loop_lag recent / worst	69ms / 751ms；170ms / 1213ms	同上	recent=最近一次 0.25s 探针；worst=进程启动以来含启动期
network api rx/tx	319 / 4629 字节每秒	约 09:06Z	仅 HTTP 载荷（Content-Length 计），无 WS/health/流式
uplink	null	同上	设计即 None，不可读不出 0
delivery unsent / dead	139 / 0	两次相同	全库计数，无 receiver 过滤
oldest_unsent_at	2026-09-24T02:20:13Z	—	无 attempts 过滤
项目 usage 累计	input 28,387,485,990； output 252,925,865； turns 24,209	约 09:08Z	累计量，非速率

两次 usage 快照差分（约一小时区间，起止时刻不精确）： $\Delta\text{turns}=7$ 。只说明「窗口内落账的归属单元约 7 个」，推不出并发高低：turns 是 distinct turn_id 归属计数（repositories.py:87-112），订阅链路 30s 近实时落账、网关链路轮末才落，在跑长轮在差分里看不到。

delivery_unsent 139 的代码级解释

三个查询三种过滤 (gaps.py:186-218、integrations.py:139-157)：计数不带 receiver 过滤；重发任务要求 receiver_id IS NOT NULL (ledger.resend_unsent, 60s 一轮、每轮 200、最老优先、上限 5 次)；最老时间不带 attempts 过滤。receiver_id 为 NULL 的行永不重试、永不成死信——与「139 未发、0 死信、最老 09-24」完全自治。NULL 行有多少、该不该发：需要只读库权限，unknown。

任务快照备份 500 (task 4db75afb, 连续三次)

路径: post-commit → cheese sync → git bundle (只含不在 origin/main 的对象) → PUT /projects/{id}/git/tasks/{id}/snapshots/{sha} (客户端超时 120s) → 512MiB 上限、digest/magic 校验失败是 422 → R2 上传带服务端 asyncio.timeout(120), 超时/上传错误未捕获 → 500。

代码层能确定的只有一点: 校验失败 (digest/magic、超 512MiB) 返回 422 而不是 500, 所以 500 来自服务端未被捕获的异常。handler 内部还有临时文件、流式写、digest、数据库、private_storage、flush 等多处都可能抛, 实际异常类型没读到, 落在哪一处 unknown。「大 bundle 超时」「部署窗口」都只是待证假设。push (forge) 与备份 (R2) 是两条独立路径, push 成功、备份 500 不矛盾。要钉死需要 dev 机后端日志异常类型 + worktree cheese-sync.log 时间戳 (都要 sudo)。

长上下文整理样本 (话题 422, 2026-09-30)

转录里三条「对话太长, 正在整理上下文; 整理完会接着处理, 期间不会回复」: 09:43:38.530Z、09:54:12.966Z、09:55:36.777Z。status 无 live_turn; root 保留远端 506ca48a, 未重启。

语义 (events.py:118-130、prompt.py:343-366、hook_stream.py:385-505): 每条 = 一次 compaction 开始; 一轮内原位复写同一块; 结束变成「上下文已整理, 接着处理」或「上下文整理没有完成」+原因; 轮在整理中结束 → 「会话在整理完成前结束了」。发生在会话宿主, 与工作机无关。三条消息是三次开始, 间隔相加不是模型生成耗时。

在场证据 (2026-09-30 10:27-10:30Z): 同一轮 (d191d12f...) 10:27:06Z、10:28:09Z 继续新增 running 提示, 提示块 meta.at 更新到 10:30:10Z; 同时 status.turn/live_turn 为 null, GET /agent/control 返回 id:null、connected:false、controls:[]。这组读数推不出「远程会话已死」, 也推不出「整理已完成」: control 空是路由在 runtime 为 None 时的兜底, ComputePool 未在话题属主候选里选出 SessionControls (可能没有或陈旧的后端 handle, 也可能选择口径如此); 该入口是房间级, 没有席位枚举, 也没有直接 ping runner 的口子。普通鉴权 API 找不到对真实 compaction attempt 的在场核验手段, 标 unknown; 不借内部 execution token、不开中央宿主 shell。这是状态可观测缺口。另: 当前 main (e173aa4, 采样 SHA 之后) 注册集仍只有 ClaudeCode, Mimo 只是 model 标签, 不用未注册骨架解释现场。

first_output_s 语义 (本采样版本)

本采样版本 (9d035d8) 注册集里只有 ClaudeCode, 通道一律包在 CentralChannel 外 (compute.py:438-468), provisions_machine=False (central_provider.py:51), 所以 chat.py:4149 的「云端准备等待」分支对当前注册集不触发。cloud_provider.py 里 provisions_machine=True 的类不在本部署注册集内, 不是真实分支, 不能拿它解释现场。

计时边界：t0 在 `_execute` 开始 (runtime.py:2336)，`first_output_s` 记在第一个 `assistant_block` 到达时 (runtime.py:2566-2568)。t0 之前还有 `admission` (`_wait_to_start` / `_admit`, runtime.py:2156-2167)：项目并发上限排队、额度闸都在那段。所以 `first_output_s` 含执行段内的组装、平台准备和模型首块，但不含 `admission` 排队；用户的实际等候 = `admission` + `first_output_s`。它不是 TTFT，也不是用户等待全长。

未决项（按需要谁排）

1. dev 机 sudo：metering-proxy gateway_request_timing、LiteLLM spend 日志 → provider tokens/s、TTFT 唯一权威来源。
2. dev 机 sudo：4db75afb 备份 500 的异常类型 + cheese-sync.log 时间戳。
3. dev 机只读库：deliveries 表 receiver_id 分桶计数。
4. 产品：机型档位表（核数/内存）确认后补进正文。
5. wangchangxin：稳态 performance 复读（部署后指标已清零两轮）。
6. 会话内部状态可观测缺口：control 空无法区分「死 / 活 / 整理中」，是否立项补观测由平台方定。